

Genéricos en C#, List<T> e IList<T>

Tecnicatura Superior en Desarrollo de Software

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Qué es?

Los genéricos en C# son un mecanismo del compilador que permite definir clases, interfaces y métodos con parámetros de tipo, representados habitualmente por `<T>`. En lugar de hardcodear un tipo concreto, se declara un marcador de posición que se resuelve en tiempo de compilación. El compilador genera internamente una versión especializada para cada tipo utilizado, garantizando seguridad sin duplicar código.

`List<T>` es la implementación concreta más utilizada para almacenar colecciones dinámicas. `IList<T>` es la interfaz que define el contrato público de esa colección. Ambos comparten la misma sintaxis de corchetes angulares, pero cumplen roles distintos en el diseño orientado a objetos.

Analogía

Piense en un enchufe eléctrico estándar. El muro tiene una forma fija que acepta múltiples dispositivos (lámpara, cargador, ventilador). Esa forma es la **interfaz** (`IList<T>`): define qué conexiones están disponibles, sin importar qué aparato se conecte. El dispositivo que lleva dentro los cables, transformadores y componentes es la **implementación concreta** (`List<T>`). Si mañana cambia de marca, mientras respete el mismo estándar de enchufe, el sistema sigue funcionando. En C#, programar hacia la interfaz permite cambiar la herramienta interna sin romper el código que la utiliza.

Características

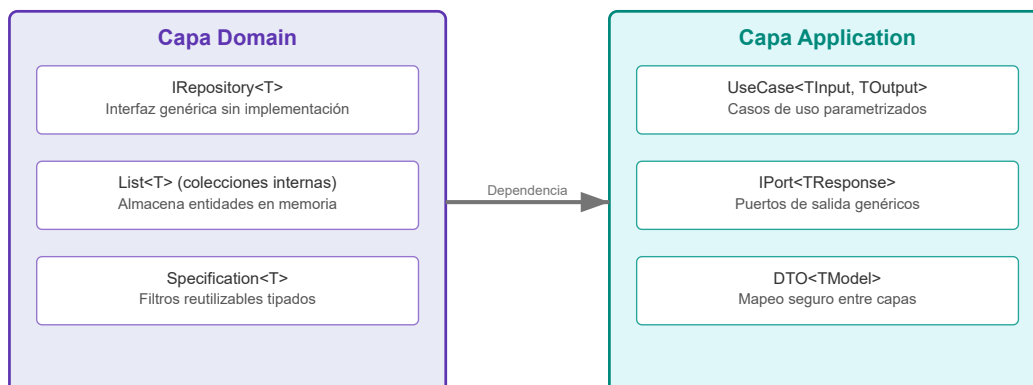
- **Seguridad de tipos en compilación:** El compilador valida operaciones antes de ejecutar, eliminando errores de conversión en tiempo de ejecución.
- **Rendimiento optimizado:** Almacena tipos de valor directamente en memoria contigua, evitando el *boxing* y *unboxing* de `object`.
- **Reutilización de código:** Una única definición de clase o método sirve para múltiples tipos sin reescribir lógica.
- **Contratos explícitos:** `ICollection<T>` garantiza métodos como `Add`, `Remove`, `IndexOf` y acceso por índice.
- **Restricciones (`where`):** Permiten limitar `T` a tipos por referencia, valor, o interfaces específicas, habilitando operaciones seguras.

¿Por qué se usa?

Antes de los genéricos, las colecciones operaban sobre `object`, obligando a conversiones manuales y aceptando fallos silenciosos en producción. Los genéricos trasladan la validación al momento de compilación, donde el IDE y el compilador detectan inconsistencias de forma inmediata.

La distinción entre `List<T>` e `ICollection<T>` surge del principio de diseño *Programar hacia contratos*. `List<T>` se usa exclusivamente para instanciar y gestionar memoria. `ICollection<T>` se usa en firmas públicas, parámetros y campos internos, desacoplando la lógica del consumidor de la implementación específica. Esto facilita pruebas unitarias, migraciones futuras y mantenimiento a largo plazo.

Arquitectura



En Clean Architecture, los genéricos fluyen desde la capa de Dominio hacia Application. Las interfaces genéricas permiten desacoplar la definición del comportamiento de su implementación concreta, cumpliendo el Principio de Dependencia.

Marco teórico

Comprender la diferencia entre `List<T>` e `ICollection<T>` requiere analizar cómo el compilador valida contratos versus implementaciones. `List<T>` es una clase sellada que gestiona un arreglo interno redimensionable, exponiendo propiedades como `Capacity`, `Count` y métodos mutadores. `ICollection<T>` es una interfaz que únicamente declara qué operaciones están disponibles, sin dictar cómo se almacenan los datos.

Cuando una firma pública devuelve o recibe `List<T>`, obliga al consumidor a depender de una implementación específica. Si el equipo decide migrar a `ObservableCollection<T>` o a una estructura personalizada, todas las referencias deben modificarse. Al usar `ICollection<T>` en la firma, el consumidor solo interactúa con el contrato. La implementación interna puede cambiar sin afectar el código externo.

Cuándo usar cada uno

Contexto	Tipo recomendado	Motivo técnico
Instanciación (<code>new</code>)	<code>List<T></code>	Requiere clase concreta para reservar memoria.
Parámetro de método	<code>IList<T></code>	Acepta cualquier implementación compatible.
Retorno de método	<code>IList<T></code>	Expone solo lo necesario, permite cambiar implementación interna.
Campo interno de clase	<code>IList<T></code>	Reduce acoplamiento, facilita pruebas unitarias con mocks.
Acceso a propiedades internas	<code>List<T></code>	Requerido para <code>Capacity</code> , <code>TrimExcess()</code> , etc.

```

1  // Instanciación: siempre usamos la clase concreta
2  var inventory = new List<Product>();
3
4  // Campo interno: preferimos el contrato para reducir acoplamiento
5  public IList<Product> Items { get; private set; } = new List<Product>();
6
7  // Método público: acepta cualquier IList<T>, no solo List<T>
8  public void ValidateBatch(IList<Product> batch)
9  {
10     foreach (var product in batch)
11     {
12         if (product.Stock < 0)
13             throw new InvalidOperationException("Stock inválido.");
14     }
15 }

```

La línea 2 muestra la instanciación obligatoria de la clase concreta. La línea 5 expone la colección mediante el contrato `IList<T>`, protegiendo el setter pero permitiendo mutación controlada. La línea 8 demuestra cómo el método acepta cualquier colección que cumpla el contrato, sin obligar al llamante a usar específicamente `List<T>`.

Restricciones con `where`

Cuando el código genérico necesita acceder a miembros específicos, se aplican restricciones. Para alumnos con base en POO, el enfoque más claro es restringir

hacia interfaces que ya definimos en el dominio. Esto evita conversiones peligrosas y mantiene el código predecible.

```
1 // Interfaz básica definida en el dominio
2 public interface INameable
3 {
4     string DisplayName { get; }
5 }
6
7 // Restricción explícita: T debe implementar INameable
8 public IList<T> FilterByName(IList<T> source, string keyword) where T : INameable
9 {
10     var result = new List<T>();
11     foreach (var item in source)
12     {
13         if (item.DisplayName.Contains(keyword))
14             result.Add(item);
15     }
16     return result;
17 }
```

La línea 8 restringe `T` a `INameable`. El compilador valida que cualquier tipo pasado posea la propiedad `DisplayName`. La línea 13 accede a esa propiedad de forma segura, sin conversiones ni validaciones manuales.

Errores comunes

ERROR COMUN

Utilizar `ArrayList` o colecciones basadas en `object` en código moderno.

```
1 var collection = new ArrayList();
2 collection.Add("Producto A");
3 collection.Add(42);
4 foreach (var element in collection)
5 {
6     Console.WriteLine(element.ToString());
7     // Boxing/unboxing implícito, s
8 }
```

Esta aproximación permite mezclar tipos arbitrarios. El compilador no puede validar operaciones específicas. En tiempo de ejecución, un `cast` incorrecto genera `InvalidCastException`.

BUENA PRACTICA

Utilizar `List<T>` con un tipo concreto explícito.

```
1 var collection = new List<string>()
2 collection.Add("Producto A");
3 // collection.Add(42); // Error CS1
4 foreach (var element in collection)
5 {
6     Console.WriteLine(element.Length);
7     // Acceso seguro a miembros de
8 }
```

El compilador rechaza tipos incompatibles antes de ejecutar. La memoria se gestiona de forma contigua para tipos de valor. Los miembros específicos están disponibles sin casting.

Qué NO hacer

QUE NO HACER

Exponer `List<T>` directamente en firmas públicas y devolver colecciones mutables sin control.

```
1 public List<Product> GetAll()
2 {
3     return this.items; // Expone la
4 }
5
6 // Cualquier llamante puede modificar
7 warehouse.GetAll().Clear(); // Rompe
```

Esta práctica rompe el encapsulamiento. El consumidor manipula la referencia interna directamente, generando efectos secundarios impredecibles.

BUENA PRACTICA

Devolver una copia de solo lectura o exponer únicamente el contrato `IList<T>` con modificación controlada.

```
1 public IList<Product> GetAll()
2 {
3     return this.items.AsReadOnly();
4 }
5
6 // O proporcionar método explícito
7 public void RemoveInvalid()
8 {
9     for (int i = this.items.Count - 1; i >= 0; i--)
10         if (!this.items[i].IsValid)
11             this.items.RemoveAt(i);
12 }
```

Se preserva la integridad del estado interno. El consumidor recibe datos sin capacidad de mutación accidental. Las modificaciones se centralizan en métodos con validación explícita.

Exponer colecciones mutables directamente viola el principio de encapsulamiento básico de POO. Siempre se debe preferir el retorno de vistas inmutables, copias defensivas o contratos `IList<T>` con control de acceso estricto. Esto garantiza que el estado del objeto permanezca consistente a lo largo de su ciclo de vida.